RSI Project

Findings to resolve

1. **Critical: administrator authentication exists only in the browser.** `src/js/admin.js:33` embeds a super-admin password and stores admin credentials in localStorage. `public/api/list_attendees.php:5` returns attendee information without server authentication; `public/api/scan.php:5` and `public/api/certificate/generate.php:289` likewise lack an admin session/role check. Hiding the dashboard does not protect these endpoints. Implement server-side admin authentication and authorization on every protected endpoint, with CSRF protection for changes. Changing the JavaScript password is not a fix.

2. **Critical: password reset allows account takeover.** `public/api/auth.php:148–189` accepts a username, stores it in the session, then changes that account's password without verifying ownership. Disable this feature until a single-use, expiring reset token delivered through a verified channel is implemented. Add authentication/reset rate limits.

4. **Blocking: supplied database export does not match the code.** `rsi_forum.sql:25` only creates the older attendees table. Current code requires `username`, `password_hash`, `is_registered`, and an `evaluations` table. Signup also omits registration fields that the old schema declares NOT NULL without defaults. Importing this export alone will not produce a working site. Obtain a fresh export from the working development database, or implement and test a migration; do not simply add three columns and assume signup works. The dump contains attendee data and DROP TABLE statements; import only into a new empty staging database. Its MySQL-specific collation is another reason to use MySQL 8 rather than assume MariaDB compatibility.

5. **Deployment paths are hard-coded.** Navigation, admin API requests and certificate URLs use `/dost-rsi-attendance-system/public/`. JavaScript lives outside public and is requested through both `/src/js/` and relative `../src/js/`. The Apache aliases below preserve those routes while exposing only public files and the JavaScript directory. Longer term, put browser assets under public and centralize the base URL.

6. **Sensitive generated files are directly accessible.** Signatures and QR images are under `public/storage`; PDFs use predictable attendee IDs under `public/generated/certificates`. Directory-listing protection does not prevent direct downloads. Before public launch, store private files outside the document root and serve them through authorized download

endpoints. The included runtime files and SQL dump contain existing data: decide deliberately whether this is a migration or a clean event setup.

7. **Dependencies need attention.** Composer audit reports two FPDI advisories affecting pinned v2.6.0: CVE-2025-54869 and CVE-2026-45802. Change the exact constraint to a maintained patched release (the audit identifies 2.6.7 as the minimum resolving both), update the lockfile in development, rerun audit and test certificates. Actual exploitability depends on parser usage; the inspected generator creates PDFs from local PNGs. Do not blindly update all production dependencies.

8. **Some functionality is unfinished or differs from README.** `public/certificate.php` displays “Coming soon”. SMTP/QR errors are logged and swallowed by `send_qr_email`, so successful registration does not prove mail delivery. Test the complete flow, including actual received mail. The QR payload points to a POST-only API: opening it with a phone's generic camera produces HTTP 405; use the application's admin scanner.